

Tutorial -2 and 3: Array

Que.1 Find the minimum and maximum value in an array.

(Asked in Microsoft and Facebook)

Examples

Input: $X[] = [4, 2, 0, 8, 20, 9, 2]$, Output: $\text{max} = 20, \text{min} = 0$

Input: $X[] = [-8, -3, -10, -32, -1]$, Output: $\text{max} = -1, \text{min} = -32$

Solution:

Solution idea and steps

- We declare and initialize variables **max** and **min** to store maximum and minimum.
- Traverse the array from $i = 1$ to $n-1$ and compare each element with **min** and **max**.
- If $(X[i] < \text{min})$, it means we have found a value smaller than minimum value till i th index. We update min with $X[i]$ i.e $\text{min} = X[i]$.
- else if $(X[i] > \text{max})$, it means we have found a value greater than maximum value till i th index. We update max with $X[i]$ i.e $\text{max} = X[i]$.
- By the end of the loop, the minimum and maximum values of the array will be stored in the variables **min** and **max**. To return this, we take extra array **res[]** of size two, where we store maximum on the first index and minimum on the second index. We return the **res** array as an output.

Q.2 Fighting the darkness

Given an array `arr[]` of size `N` representing the size of candles which reduce by 1 unit each day. The room is illuminated using the given `N` candles. Find the maximum number of days the room is without darkness.

Input: Array of size `n` with some positive value in each cell

Asked in Snapdeal

Solution: Find the maximum value in the array

Example 1:

Input: `N = 3, arr[] = {1,1,2}`

Output: 2

Explanation: The candles' length reduce by 1 in 1 day. So, at the end of day 1: Sizes would be 0 0 1, So, at end of day 2: Sizes would be 0 0 0. This means the room was illuminated for 2 days.

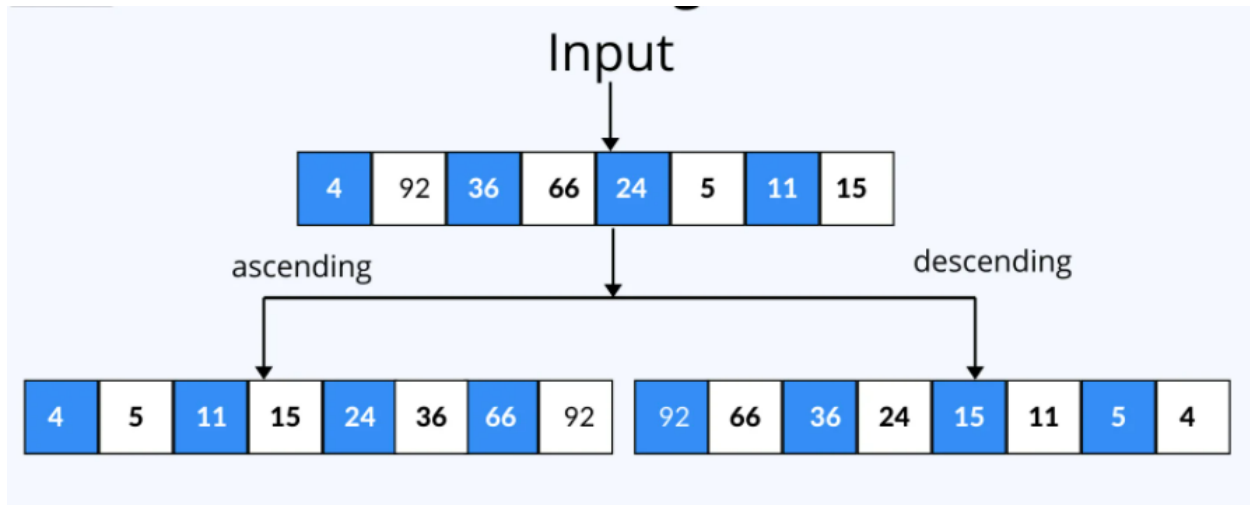
Example 2:

Input: `N = 5, arr[] = {2,3,4,2,1}`

Output: 4

Que.3 Sort the Array in an Ascending Order and Descending order.

Ascending Order:



Solution:

Problem Solution

1. Create an array of fixed size (maximum capacity), let's say 10.
2. Take n, a variable which stores the number of elements of the array, less than maximum capacity of array.
3. Iterate via for loop to take array elements as input, and print them.
4. The array elements are in unsorted fashion, to sort them, make a nested loop.
5. In the nested loop, the each element will be compared to all the elements below it.
6. In case the element is greater than the element present below it, then they are interchanged.
7. After executing the nested loop, we will obtain an array in ascending order arranged elements.

```
for (i = 0; i < n; ++i)
{
    for (j = i + 1; j < n; ++j)
    {
        if (number[i] > number[j])
        {
            a = number[i];
            number[i] = number[j];
            number[j] = a;
        }
    }
}
```

Descending Order:

Problem Solution

1. Create an array of fixed size (maximum capacity), lets say 10.
2. Take n, a variable which stores the number of elements of the array, less than maximum capacity of array.
3. Iterate via for loop to take array elements as input, and print them.
4. The array elements are in unsorted fashion, to sort them, make a nested loop.
5. In the nested loop, the each element will be compared to all the elements below it.
6. In case the element is smaller than the element present below it, then they are interchanged
7. After executing the nested loop, we will obtain an array in descending order arranged elements.

```
for (i = 0; i < n; ++i)
{
    for (j = i + 1; j < n; ++j)
    {
        if (number[i] < number[j])
        {
            a = number[i];
            number[i] = number[j];
            number[j] = a;
        }
    }
}
```

Que.4 Find the "Kth" max and min element of an array.

Asked in Amazon, Google, Microsoft, Samsung, Walmart, Cisco, Adobe, VMware, Snapdeal

Solution:

Kth Max:

Method 1

- (1) A Simple Solution is to sort the given array in descending order as method discussed above.
- (2) Print (k-1) index element of sorted array.

Method-2:

1. Find the maximum element of the array and store/make new array without this element.
2. Then find 2nd maximum element in this array and store/make new array without this element.
3. Repeat the step for kth number of times.

Kth Min:

Method 1:

- (1) A Simple Solution is to sort the given array in ascending order as method discussed above.
- (2) Print (k-1) index element of sorted array.

Method-2:

1. Find the minimum element of the array and store/make new array without this element.
2. Then find 2nd minimum element in this array and store/make new array without this element.
3. Repeat the step for kth number of times.

Note: There can be more than two method to solve above problem like using heap but we have not taught this concept in class so avoid such solution.

Que.5 Given an array `Arr[]` of `N` positive integers, find `K` largest elements from the array. The output elements should be printed in decreasing order.

Asked in Amazon, Google, Microsoft, Walmart, Samsung

Input:

`N = 5, K = 2`

`Arr[] = {12, 5, 787, 1, 23}`

Output: 787 23

Input:

`N = 7, K = 3`

`Arr[] = {1, 23, 12, 9, 30, 2, 50}`

Output: 50 30 23

Solution:

Array need to print in descending order then we have to print `k-1` element from beginning.

Que.6 Given an array which consists of only 0, 1 and 2. Sort the array without using any sorting algorithm.

Asked in Amazon, Microsoft, SAP Labs, Paytm, Adobe, Ola cab, Oyo Room, Snapdeal, Yatra.com, Samsung, Walmart

```
Input: {0, 1, 2, 0, 1, 2}
Output: {0, 0, 1, 1, 2, 2}
```

```
Input: {0, 1, 1, 0, 1, 2, 1, 2, 0, 0, 0, 1}
Output: {0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2}
```

Solution: Double Traversal (Brute Force approach)

The first approach that comes in mind is to simply count the number of 0's, 1's, and 2's. Then, place all 0's at the beginning of the array followed by 1's and 2's.

Algorithm Steps

1. Declare three variables `count_zero`, `count_ones`, `count_twos` to store the respective frequency of appearances of 0's, 1's, and 2's in the array.
2. Traverse through the array and update the frequency of 0's, 1's, and 2's.
3. Now again, traverse the array and replace first `count_zero` places of the array as 0, next `count_ones` by 1, and last `count_twos` by 2.

Note: This problem has one more solution which is more efficient but I am not mentioning here because first let them try without thinking about complexity.

Que.7 Find the Union and Intersection of the two sorted arrays not containing 1 element twice.

Asked in Google, Microsoft, SAP Labs, Paytm

The union of two arrays: is the set of all elements that are either in **A** or in **B**.

The intersection of two arrays: is the set of all elements which are common **A** or in **B**.

Example:

Example:

Array: [1,2,3,4,5]

Array1: [5,3,6,7,9]

Output:

Union = [1,2,3,4,5,6,7,9]

Intersection = [3,5]

Solution:

(***Note:** Array should not contain 1 element twice.)

Union

- Initialize the two arrays with random values.
- Create an empty array called union_result.
- Iterate over the first array and add every element to it.
- Iterate over the second array and add element if it is not present in union_result array.
- Print the union_result array.

Intersection

- Initialize the two arrays with random values.
- Create an empty array called intersection_result.
- Iterate over the first array and add element if it is present in second array.
- Print the intersection_result array.

Que.8 Segregate 0s and 1s in an array

Asked in Amazon, Yahoo

You are given an array of 0s and 1s in random order. Segregate 0s on left side and 1s on right side of the array [Basically you have to sort the array]. Traverse array only once.

Input array = [0, 1, 0, 1, 0, 0, 1, 1, 1, 0]

Output array = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

Solution:

Method 1 (Count 0s or 1s)

1) Count the number of 0s. So let's understand with an example we have an array **arr = [0, 1, 0, 1, 0, 0, 1]** the size of the array is **7** now we will traverse the entire array and find out the number of zeros in the array, In this case the number of zeros is **4** so now we can easily get the number of **Ones** in the array by **Array Length – Number Of Zeros**.

2) Once we have counted, we can fill the array first we will put the **zeros** and then **ones** (we can get number of ones by using above formula).

Method 2 (Use two indexes to traverse)

Maintain two indexes. Initialize the first index *left* as 0 and second index *right* as n-1.

Do following while *left* < *right*

- a) Keep incrementing index *left* while there are 0s at it
- b) Keep decrementing index *right* while there are 1s at it
- c) If *left* < *right* then exchange arr[*left*] and arr[*right*]

Que.9 Move all zeroes to end of array.

Given an array of random numbers, Push all the zero's of a given array to the end of the array. For example, if the given arrays is {1, 9, 8, 4, 0, 0, 2, 7, 0, 6, 0}, it should be changed to {1, 9, 8, 4, 2, 7, 6, 0, 0, 0, 0}. The order of all other elements should be same.

Asked in Amazon, Paytm, LinkedIn, Bloomberg, Microsoft, Sap Lab

Input : arr[] = {1, 2, 0, 4, 3, 0, 5, 0};

Output : arr[] = {1, 2, 4, 3, 5, 0, 0};

Input : arr[] = {1, 2, 0, 0, 0, 3, 6};

Output : arr[] = {1, 2, 3, 6, 0, 0, 0};

Solution:

1. Brute Force Approach : Using an auxiliary space

We can create an auxiliary array B[] of size n which will be used to store all the non-zero elements as we linearly traverse A[]. Let the number of non-zero elements be x. Now fill the remaining n-x elements with zeroes and return B[].

```
int[] moveZeroes(int A[], int n)
{
    int B[n]
    int j = 0
    for (i = 0 to n-1)
    {
        if (A[i] != 0)
        {
            B[j] = A[i]
            j = j + 1
        }
    }

    while (j < n)
    {
        B[j] = 0
        j = j + 1
    }
    return B
}
```

2. Two pointer approach I: In-place using two traversals

A two-pointer approach could be used for these types of questions. A pointer **i** to linearly traverse **A[]** and a pointer **j** to track the non-zero element. After the whole process, all elements in **A[0...j]** are non-zero elements.

Solution Steps

1. Initialize **j = 0**.
2. Run a for loop from **i = 0** to **n-1** to traverse **A[]**.
3. At any step of iteration, if (**A[i] != 0**) then update **A[j] = A[i]** and increment **j** by 1. Here we are shifting each non-zero element to the start of **A[]**.
4. After completing the loop, traverse **A[]** from **j** to **n-1** and fill it with zeroes.

```
int[] moveZeroes(int A[], int n)
{
    int j = 0
    for (i = 0 to n-1)
    {
        if (A[i] != 0)
        {
            A[j] = A[i]
            j = j + 1
        }
    }
    while (j < n)
    {
        A[j] = 0
        j = j + 1
    }
    return A
}
```

Que.10 Trapping Rain Water

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it is able to trap after raining.

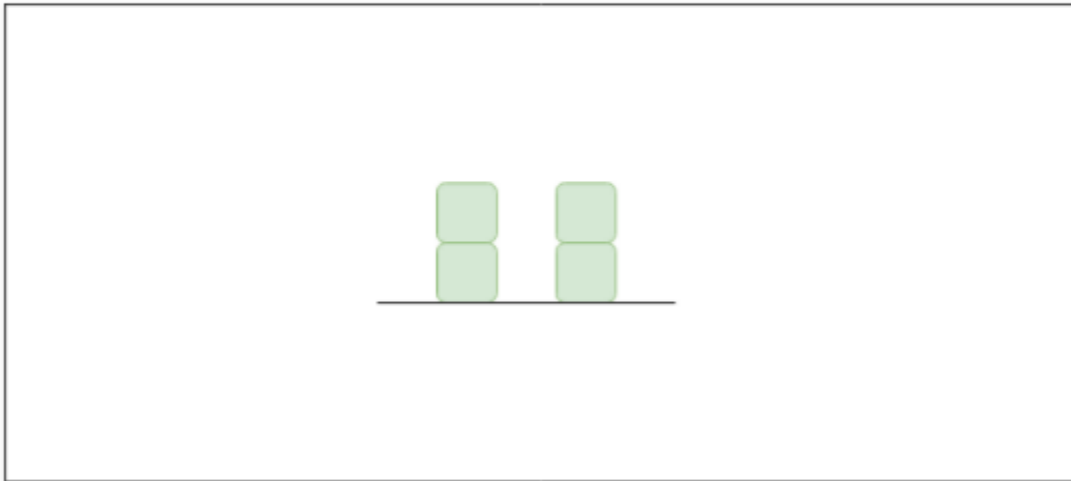
Asked in Amazon, Google, Microsoft

Input: `arr[] = {2, 0, 2}`

Output: 2

Explanation:

The structure is like below



We can trap 2 units of water in the middle gap.

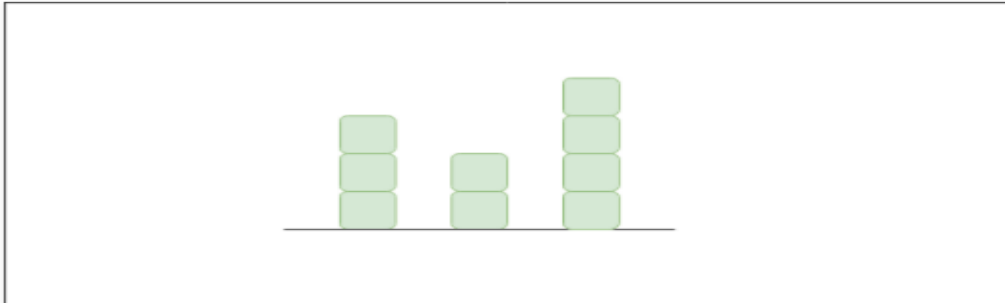
We can trap 2 units of water in the middle gap.

Input: arr[] = {3, 0, 2, 0, 4}

Output: 7

Explanation:

Structure is like below



We can trap "3 units" of water between 3 and 2, "1 unit" on top of bar 2 and "3 units" between 2 and 4. See below diagram also.

Solution:

Method 1 :

This is a simple solution to the above problem.

- **Approach:** The idea is to traverse every array element and find the highest bars on the left and right sides. Take the smaller of two heights. The difference between the smaller height and height of the current element is the amount of water that can be stored in this array element.
- **Algorithm:**
 1. Traverse the array from start to end.
 2. For every element, traverse the array from start to that index and find the maximum height (a) and traverse the array from the current index to end, and find the maximum height (b).
 3. The amount of water that will be stored in this column is $\min(a, b) - \text{array}[i]$, add this value to the total amount of water stored
 4. Print the total amount of water stored.

Que.11 Given an array of n integers where each value represents the number of chocolates in a packet. Each packet can have a variable number of chocolates. There are m students, the task is to distribute chocolate packets such that:

1. Each student gets one packet.
2. The difference between the number of chocolates in the packet with maximum chocolates and packet with minimum chocolates given to the students is minimum.

Asked in Amazon, Paytm, Walmart, Adobe

Examples:

Input : $\text{arr}[] = \{7, 3, 2, 4, 9, 12, 56\}$, $m = 3$

Output: Minimum Difference is 2

Explanation:

We have seven packets of chocolates and we need to pick three packets for 3 students. If we pick 2, 3 and 4, we get the minimum difference between maximum and minimum packet sizes.

Input : $\text{arr}[] = \{3, 4, 1, 9, 56, 7, 9, 12\}$, $m = 5$

Output: Minimum Difference is 6

Explanation:

The set goes like 3,4,7,9,9 and the output is $9-3 = 6$

Solution:

A **simple solution** is to generate all subsets of size m of $\text{arr}[0..n-1]$. For every subset, find the difference between the maximum and minimum elements in it. Finally, return the minimum difference.

An **efficient solution** is based on the observation that to minimize the difference, we must choose consecutive elements from a sorted packet. We first sort the array $\text{arr}[0..n-1]$, then find the subarray of size m with the minimum difference between the last and first elements.

Below image is a dry run of the above approach:

Initial array

7	3	2	4	9	12	56
---	---	---	---	---	----	----

 , $m = 3$

After Sorting

2	3	4	7	9	12	56
---	---	---	---	---	----	----

 , $\text{min_diff} = \text{INT_MAX}$
 $m = 3$

Step 1:

2	3	4	7	9	12	56
---	---	---	---	---	----	----

 , $m = 3$, $\text{min_diff} = \text{INT_MAX}$
 $\uparrow \quad \quad \uparrow$
 $i \quad \quad i+m-1$

Here, $a[i + m - 1] - a[i] < \text{min_diff}$. update min_diff
 $\text{min_diff} = a[i + m - 1] - a[i]$

Step 2:

2	3	4	7	9	12	56
---	---	---	---	---	----	----

 , $m = 3$, $\text{min_diff} = 2$
 $\uparrow \quad \quad \uparrow$
 $i \quad \quad i+m-1$

Here, $a[i + m - 1] - a[i] > \text{min_diff}$. Do nothing

Step 3:

2	3	4	7	9	12	56
---	---	---	---	---	----	----

 , $m = 3$, $\text{min_diff} = 2$
 $\uparrow \quad \quad \uparrow$
 $i \quad \quad i+m-1$

Here, $a[i + m - 1] - a[i] > \text{min_diff}$. Do nothing

Step 4:

2	3	4	7	9	12	56
---	---	---	---	---	----	----

 , $m = 3$, $\text{min_diff} = 2$
 $\uparrow \quad \quad \uparrow$
 $i \quad \quad i+m-1$

Here, $a[i + m - 1] - a[i] > \text{min_diff}$. Do nothing

Step 5:

2	3	4	7	9	12	56
---	---	---	---	---	----	----

 , $m = 3$, $\text{min_diff} = 2$
 $\uparrow \quad \quad \uparrow$
 $i \quad \quad i+m-1$

Here, $a[i + m - 1] - a[i] > \text{min_diff}$. Do nothing

Que 12. Given arrival and departure times of all trains that reach a railway station. Find the minimum number of platforms required for the railway station so that no train is kept waiting.

Note:

Consider that all the trains arrive on the same day and leave on the same day. Arrival and departure time can never be the same for a train but we can have arrival time of one train equal to departure time of the other. At any given instance of time, same platform cannot be used for both departure of a train and arrival of another train. In such cases, we need different platforms.

Asked in Google, Amazon, Paytm, Walmart, Adobe, Hike, D-E-Shaw, Boomerang Commerce

```
Trains arrival    = { 2.00, 2.10, 3.00, 3.20, 3.50, 5.00 }  
Trains departure = { 2.30, 3.40, 3.20, 4.30, 4.00, 5.20 }
```

```
The minimum platforms needed is 2
```

```
The train arrived at 2.00 on platform 1  
The train arrived at 2.10 on platform 2  
The train departed at 2.30 from platform 1  
The train arrived at 3.00 on platform 1  
The train departed at 3.20 from platform 1  
The train arrived at 3.20 on platform 1  
The train departed at 3.40 from platform 2  
The train arrived at 3.50 on platform 2  
The train departed at 4.00 from platform 2  
The train departed at 4.30 from platform 1  
The train arrived at 5.00 on platform 1  
The train departed at 5.20 from platform 1
```

```
arr[] = {9:00, 9:40, 9:50, 11:00, 15:00, 18:00}
dep[] = {9:10, 12:00, 11:20, 11:30, 19:00, 20:00}
```

All events sorted by time.

Total platforms at any time can be obtained by subtracting total departures from total arrivals by that time.

Time	Event Type	Total Platforms Needed at this Time
9:00	Arrival	1
9:10	Departure	0
9:40	Arrival	1
9:50	Arrival	2
11:00	Arrival	3
11:20	Departure	2
11:30	Departure	1
12:00	Departure	0
15:00	Arrival	1
18:00	Arrival	2
19:00	Departure	1
20:00	Departure	0

Minimum Platforms needed on railway station = Maximum platforms
needed at any time
= 3

Solution:

- **Approach:** The idea is to take every interval one by one and find the number of intervals that overlap with it. Keep track of the maximum number of intervals that overlap with an interval. Finally, return the maximum value.

Algorithm:

1. Sort the arrival and departure times of trains.
2. Create two pointers $i=0$, and $j=0$ and a variable to store *ans* and current count *plat*
3. Run a loop while $i < n$ and $j < n$ and compare the i th element of arrival array and j th element of departure array.
4. If the arrival time is less than or equal to departure then one more platform is needed so increase the count, i.e. $plat++$ and increment i
5. Else if the arrival time greater than departure then one less platform is needed so decrease the count, i.e. $plat--$ and increment j
6. Update the ans, i.e $ans = \max(ans, plat)$.